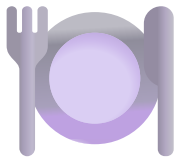


# Kiekje

Zo werkt de app — heel simpel uitgelegd



Uitleg van alle code, met plaatjes, een spiekbriefje voor de presentatie  
en antwoorden op vragen die de docent kan stellen.

Tip: in de oranje vakjes staat de simpele uitleg, in de groene vakjes staat wat je hardop kunt zeggen.

# Inhoud

---

- |                                      |                                            |
|--------------------------------------|--------------------------------------------|
| 1. Wat is Kiekje?                    | 9. De voorraadkast: de database            |
| 2. Het grote plaatje: een restaurant | 10. Zo gaat het stap voor stap             |
| 3. Wat kan de app allemaal?          | 11. Veiligheid: de sloten op de deur       |
| 4. De mappen: waar staat wat?        | 12. Testen: de robot die alles uitprobeert |
| 5. De eetzaal: de frontend           | 13. De tekeningen (diagrammen)             |
| 6. React: de zoekpagina van LEGO     | 14. Spiekbriefje voor de presentatie       |
| 7. De ober: de API                   | 15. Vragen die de docent kan stellen       |
| 8. De keuken: de backend (OOP)       | 16. Woordenlijst                           |

## 1. Wat is Kiekje?

---

Kiekje is een app zoals **Instagram**. Je kunt foto's delen met je vrienden. Je kunt foto's leuk vinden, erop reageren, andere mensen volgen en berichtjes sturen.



### Heel simpel

Kiekje is een groot **fotoboek dat iedereen samen maakt**. Iedereen mag er foto's in plakken. En iedereen mag bij andermans foto een hartje tekenen of iets liefs schrijven.

De app is gemaakt met:

- **HTML** — de stenen van het huis (wat er op het scherm staat)
- **CSS** — de verf en het behang (kleuren, vormen, hoe het eruitziet)
- **JavaScript** — de elektriciteit (knoppen die iets doen)
- **React** — LEGO-blokjes voor de zoekpagina
- **PHP** — de kok in de keuken (de server)
- **MySQL** — de voorraadkast (de database, waar alles bewaard wordt)

## 2. Het grote plaatje: een restaurant

---

De makkelijkste manier om de app te snappen: stel je een **restaurant** voor.

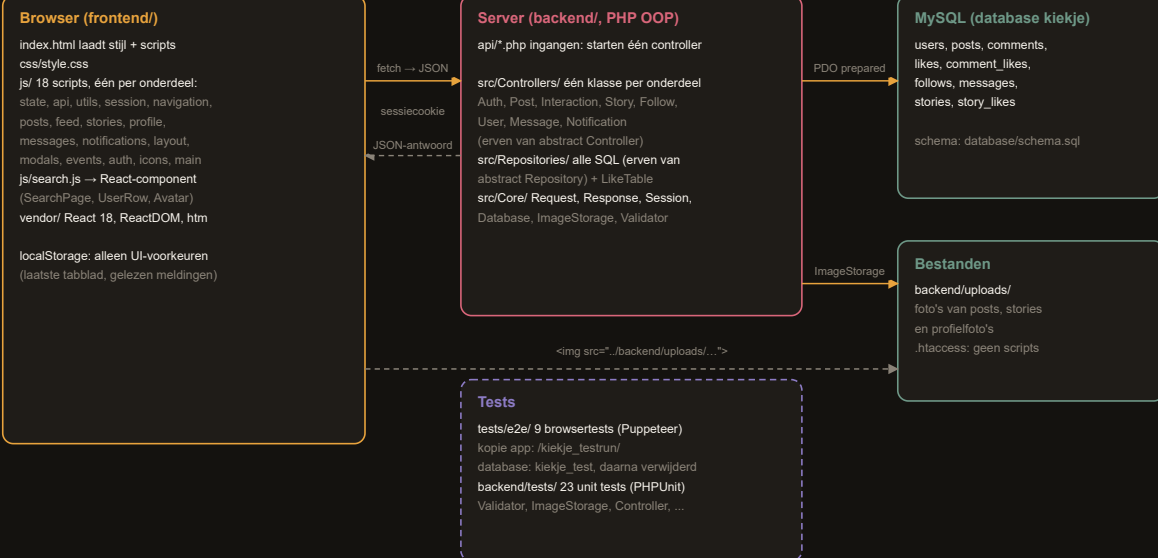
| In het restaurant                                                                                                                                         | In Kiekje                                          | Waar in de code?                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|----------------------------------|
|  <b>De eetzaal:</b> waar jij zit en de menukaart ziet                    | De <b>frontend:</b> wat je in de browser ziet      | <code>frontend/</code>           |
|  <b>De ober:</b> brengt jouw bestelling naar de keuken en het eten terug | De <b>API:</b> berichtjes tussen browser en server | <code>backend/api/</code>        |
|  <b>De keuken:</b> daar wordt het eten gemaakt en gecontroleerd          | De <b>backend:</b> PHP-code op de server           | <code>backend/src/</code>        |
|  <b>De voorraadkast:</b> waar al het eten bewaard wordt                  | De <b>database</b> (MySQL)                         | <code>database/schema.sql</code> |
|  <b>Het fotoalbum aan de muur</b>                                        | De map met <b>geüploade foto's</b>                 | <code>backend/uploads/</code>    |

### Heel simpel

Jij zit aan tafel en zegt: *"Ik wil deze foto leuk vinden!"* De ober loopt naar de keuken. De kok kijkt of jij wel echt een klant bent (ingelogd). Dan pakt de kok het boek uit de voorraadkast, zet er een hartje bij, en de ober komt terug: *"Gelukt! Nu heeft de foto 5 hartjes."*

## Architectuur — Kiekje

Hoe frontend, API, database en bestandsopslag samenwerken



De frontend is één pagina (geen herladen): render() tekent het scherm opnieuw uit de toestand in state.js. Alle gegevens komen via de API uit MySQL; de server controleert bij elk verzoek de PHP-sessie, zodat je alleen je eigen posts, stories en reacties kunt wijzigen.

Het architectuurdiagram: eetzaal (browser), keuken (server), voorraadkast (MySQL) en fotoalbum (uploads).

### Zeg dit

"Kiekje bestaat uit drie lagen. De **frontend** is wat je ziet in de browser. Die praat via een **API** met de **backend** in PHP. De backend bewaart alles in een **MySQL-database** en slaat foto's op in een map."

## 3. Wat kan de app allemaal?

| Onderdeel                                                                                                                                                                                   | Wat kun je doen?                                                                       |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
|  Account                                                                                                   | Registreren, inloggen, uitloggen, naam en bio veranderen                               |
|  Profielfoto                                                                                               | Foto kiezen, inzoomen, verschuiven, draaien. Wie niks kiest krijgt een grijs poppetje. |
|  Posts                                                                                                     | Foto plaatsen met tekst en plaats, bewerken, weggooien, link delen                     |
|  Liken                                                                                                     | Op het hartje klikken, of 2× snel op de foto tikken (dan komt er een groot hartje)     |
|  Reacties                                                                                                | Reageren, alle reacties bekijken, reacties liken, reacties weggooien                   |
|  Stories                                                                                                 | Foto die <b>24 uur</b> blijft staan, liken, erop reageren via een berichtje            |
|  Volgen                                                                                                  | Mensen volgen en ontvolgen, zien wie jou volgt, voorgestelde vrienden                  |
|  Berichten                                                                                               | Privé-berichtjes sturen (DM's), zien of ze gelezen zijn ("Gezien")                     |
|  Meldingen                                                                                               | Zien wie jouw foto leuk vond, reageerde of jou ging volgen                             |
|  Zoeken                                                                                                  | Andere mensen zoeken terwijl je typt                                                   |
|  Telefoon +  computer | Op de telefoon een balk onderin, op de computer een balk links                         |

## 4. De mappen: waar staat wat?

Alle code staat in de map `C:\xampp\htdocs\kiekje\kiekje\`. Dit zijn de laden van de kast:

```
kiekje/
├── frontend/      ← de eetzaal: wat je ziet (HTML, CSS, JavaScript, React)
│   ├── index.html ← de voordeur: dit bestand open je in de browser
│   ├── css/       ← de verf: hoe alles eruit ziet
│   ├── js/        ← 18 bestanden: wat de knoppen doen
│   └── vendor/     ← React (geleende LEGO-doos)
├── backend/       ← de keuken: PHP op de server
│   ├── api/       ← de deurtjes waar de ober doorheen loopt
│   ├── src/       ← de koks (klassen): Controllers, Repositories, Core
│   ├── uploads/   ← het fotoalbum
│   └── tests/      ← kleine tests voor de koks
├── database/      ← de tekening van de voorraadkast (schema.sql)
├── tests/e2e/     ← de robot die de hele app uitprobeert
├── diagrams/      ← de tekeningen (ERD, klassendiagram, ...)
├── docs/          ← uitleg (dit document, API, architectuur)
└── vue/           ← een los voorbeeldje in Vue
```

## 5. De eetzaal: de frontend

De frontend is **alles wat je ziet** in de browser. Het is **één pagina** ( `index.html` ). Als je op een knop drukt, laadt de pagina niet opnieuw: het scherm verandert gewoon. Dat heet een **single-page app**.

### Het geheim: het "geheugen" en de "tekenaar"



#### Heel simpel

De app heeft een **geheugen** ( `state.js` ). Daarin staat: wie ben jij, op welke pagina ben je, welke foto's zijn er.

Dan is er een **tekenaar** ( `render()` ). Die kijkt in het geheugen en **tekent het hele scherm**.

Druk je op een knop? Dan verandert het geheugen, en de tekenaar tekent alles opnieuw. Net als een tekening die je gumt en opnieuw maakt.

1. **Geheugen** ( `state.js` ): `me` = wie ben ik, `tab` = welke pagina, `feedPosts` = de foto's.

2. **Tekenen** ( `render()` in `layout.js` ): maakt van het geheugen HTML.
3. **Knoppen aansluiten** ( `events()` in `events.js` ): zegt tegen elke knop wat hij moet doen.
4. Iemand klikt → de knop vraagt iets aan de server → het geheugen verandert → `render()` tekent opnieuw.

## De 18 JavaScript-bestanden (elk bestand is één kamertje)

| Bestand                       | Wat doet het? (simpel)                                                                                                                                                   |
|-------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>state.js</code>         |  Het geheugen van de app                                                                |
| <code>api.js</code>           |  De ober: één functie <code>api()</code> die alle bestellingen naar de server brengt    |
| <code>utils.js</code>         |  Gereedschapskist: tijd schrijven ("5 min"), foto's kleiner maken, meldingen laten zien |
| <code>session.js</code>       |  Wie is er ingelogd? Klopt dat nog met de server?                                       |
| <code>navigation.js</code>    |  Van pagina wisselen en de juiste dingen ophalen                                        |
| <code>posts.js</code>         |  Liken, reageren, foto plaatsen, bewerken, weggooien                                   |
| <code>feed.js</code>          |  De tijdlijn tekenen: fotokaartjes, reacties, dubbeltik-hartje                        |
| <code>stories.js</code>       |  De rondjes bovenaan (stories)                                                        |
| <code>profile.js</code>       |  Profiel, volgen, profielfoto met bijsnijden                                          |
| <code>messages.js</code>      |  Berichtjes (DM's)                                                                    |
| <code>notifications.js</code> |  Meldingen                                                                            |
| <code>search.js</code>        |  Zoeken (gemaakt met <b>React</b> )                                                   |
| <code>auth.js</code>          |  Het inlog- en registratiescherf                                                      |
| <code>icons.js</code>         |  De plaatjes van de knoppen (huisje, hartje, belletje)                                |
| <code>layout.js</code>        |  De balken (links, boven, onder) en de tekenaar <code>render()</code>                 |
| <code>modals.js</code>        |  Pop-up-venstertjes (nieuwe foto, bericht, story)                                     |
| <code>events.js</code>        |  Sluit alle knoppen aan                                                               |
| <code>main.js</code>          |  De aan-knop: start de app                                                            |

## Voorbeeld: de ober ( `api.js` )

```
// breng een bestelling naar de keuken en wacht op het antwoord
async function api(path, body) {
    const res = await fetch(API + path, { ... }); // ober loopt naar de keuken
    const data = await res.json();               // ober komt terug met het
eten
    if (res.status === 401) syncUser(null);        // "u bent geen klant meer"
→ naar inloggen
    if (data.error) throw new Error(data.error); // iets mis? vertel het
    return data;
}
```



### Heel simpel

`fetch` = de ober loopt naar de keuken. `await` = even **wachten** tot de ober terug is.  
`401` betekent: "Jij bent niet (meer) ingelogd."

## Waarom `esc()` ? (tegen stoute tekst)

Als iemand als reactie `<script>` typt, zou de browser dat kunnen uitvoeren als code. De functie `esc()` maakt er gewone letters van. Zo kan niemand stoute code in de app stoppen. Dit heet **XSS voorkomen**.

## 6. React: de zoekpagina van LEGO

De zoekpagina ( `frontend/js/search.js` ) is gebouwd met **React**.



### Heel simpel

React is een doos **LEGO**. Je bouwt kleine blokjes (**componenten**) en klikt ze in elkaar.



`Avatar` = het blokje met een profielfotootje



`UserRow` = een regeltje met één persoon (foto + naam)



`SearchPage` = de hele zoekpagina, gemaakt van al die blokjes

Elk blokje heeft een eigen **geheugentje** ( `useState` ): wat heb je getypt, wie is er gevonden.

```
function SearchPage() {
  const [query, setQuery] = useState(''); // geheugentje: wat is er
  gettypt?
  const [results, setResults] = useState(null); // geheugentje: wie is er
  gevonden?

  useEffect(() => { // als "query" verandert...
    // ... wacht 250 ms en vraag dan de server: wie heet zo?
  }, [query]);

  return html`<input value=${query} onChange=... /> ...`;
}
```

- **useState** = een geheugentje. Verandert het? Dan tekent React het blokje vanzelf opnieuw.
- **useEffect** = "doe iets als dit verandert". Hier: zoeken als je typt.
- We wachten **250 milliseconden** na het typen, zodat we niet bij elke letter de server lastigvallen.
- **htm** is een klein hulpje waarmee we React-code kunnen schrijven zonder extra programma's (geen "build-stap").



### Zeg dit

"De zoekpagina is een React-component. Hij bestaat uit kleinere componenten:

`SearchPage` , `UserRow` en `Avatar` . Met `useState` onthoudt hij de zoekterm en de resultaten, en met `useEffect` zoekt hij live mee terwijl je typt."

## 7. De ober: de API

De API is de **lijst met dingen die je aan de keuken mag vragen**. Elk deurtje is een PHP-bestand in `backend/api/`:

| Deurtje                        | Waarvoor?                      | Voorbeeld-vraag              |
|--------------------------------|--------------------------------|------------------------------|
| <code>auth.php</code>          | Inloggen, registreren, profiel | "Log mij in"                 |
| <code>posts.php</code>         | Foto's                         | "Geef mij alle foto's"       |
| <code>interactions.php</code>  | Likes en reacties              | "Zet een hartje bij foto 12" |
| <code>stories.php</code>       | Stories                        | "Welke stories zijn er?"     |
| <code>follows.php</code>       | Volgen                         | "Ik wil Lisa volgen"         |
| <code>users.php</code>         | Zoeken en profielen            | "Wie heet 'tim'?"            |
| <code>messages.php</code>      | Berichtjes                     | "Stuur 'hoi' naar Noor"      |
| <code>notifications.php</code> | Meldingen                      | "Wat is er nieuw voor mij?"  |

De ober praat in **JSON**: dat is een briefje met labeltjes, bijvoorbeeld:

```
{ "success": true, "liked": true, "like_count": 5 } ← "gelukt, je vindt hem leuk, er zijn nu 5 hartjes"
```

Gaat er iets mis? Dan geeft de server een **foutcode**: `401` = niet ingelogd, `403` = dat mag jij niet, `404` = bestaat niet, `422` = verkeerd ingevuld. Alle vragen en antwoorden staan in `docs/api.md`.

## 8. De keuken: de backend (OOP)

De backend is geschreven in **PHP** en is **objectgeoriënteerd (OOP)**. Wat betekent dat?

### Heel simpel: koekjes!

Een **klasse** is een **koekjesvorm** 🍪. Een **object** is een **koekje** dat je met die vorm maakt.

Van één vorm kun je veel koekjes maken. In de code: `new PostController()` = "maak een koekje met de vorm PostController".

### De drie soorten koks

| Soort                                                                                                      | Map                            | Wat doen ze? (simpel)                                                                                                                                                                                                                                                |
|------------------------------------------------------------------------------------------------------------|--------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <br><b>Controllers</b>    | <code>src/Controllers/</code>  | De <b>chef-koks</b> . Ze krijgen de bestelling, controleren of alles klopt en bepalen wat er moet gebeuren. Eén chef per onderdeel (posts, stories, berichten...).                                                                                                   |
| <br><b>Repositories</b> | <code>src/Repositories/</code> | De <b>voorraadbeheerders</b> . Alleen zij mogen in de voorraadkast (database). Ze pakken dingen of zetten dingen terug.                                                                                                                                              |
| <br><b>Core</b>         | <code>src/Core/</code>         | Het <b>gereedschap</b> van de keuken: de bestelling lezen ( <code>Request</code> ), het antwoord sturen ( <code>Response</code> ), wie is er binnen ( <code>Session</code> ), foto's opbergen ( <code>ImageStorage</code> ), controleren ( <code>Validator</code> ). |

### Overerving: kinderen lijken op hun ouders

#### Heel simpel

Er is één **oma-kok**: `Controller`. Zij weet hoe je netjes werkt: fouten opruimen, kijken wie er ingelogd is.

Alle andere koks zijn haar **kleinkinderen**: `PostController`, `StoryController`, enzovoort. Ze hebben die kunstjes van oma **geërfd**, dus ze hoeven het niet opnieuw te leren. Dat heet **overerving** ( `extends` ).

```

abstract class Controller {                                // de oma-kok (je kunt haar zelf niet
"bestellen": abstract)
    public function run() { ... }                          // werkt netjes, vangt fouten op
    abstract protected function handle();                 // "elk kleinkind maakt zijn eigen
gerecht"
    protected function currentUser() { ... }               // wie is er ingelogd?
}

class PostController extends Controller {                  // kleinkind: erft alles van oma
    protected function handle() { ... }                   // zijn eigen gerecht: foto's
}

```

## De vier OOP-woorden die je moet kennen

| Woord               | Simpel                                                   | Waar in Kiekje?                                                                               |
|---------------------|----------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>Overerving</b>   | Kind erft kunstjes van ouder                             | 8 controllers erven van <code>Controller</code> , 8 repositories van <code>Repository</code>  |
| <b>Abstractie</b>   | Een "plan" dat je niet zelf gebruikt, alleen de kinderen | <code>abstract class Controller</code> , <code>abstract handle()</code>                       |
| <b>Encapsulatie</b> | Een doos met een slot: niet iedereen mag erin            | <code>private</code> en <code>protected</code> : alleen de repositories mogen bij de database |
| <b>Polymorfisme</b> | Dezelfde vraag, elk kind doet het op zijn eigen manier   | <code>run()</code> roept bij elke controller een andere <code>handle()</code> aan             |

Nog een slim ding: `LikeTable`. Liken werkt bij foto's, reacties én stories precies hetzelfde. Dus hebben we één klasse gemaakt die we drie keer **hergebruiken**, in plaats van drie keer dezelfde code te schrijven.

## Hoe een deurtje werkt

Een deurtje in `backend/api/` is heel klein. Het roept alleen de juiste chef-kok:

```

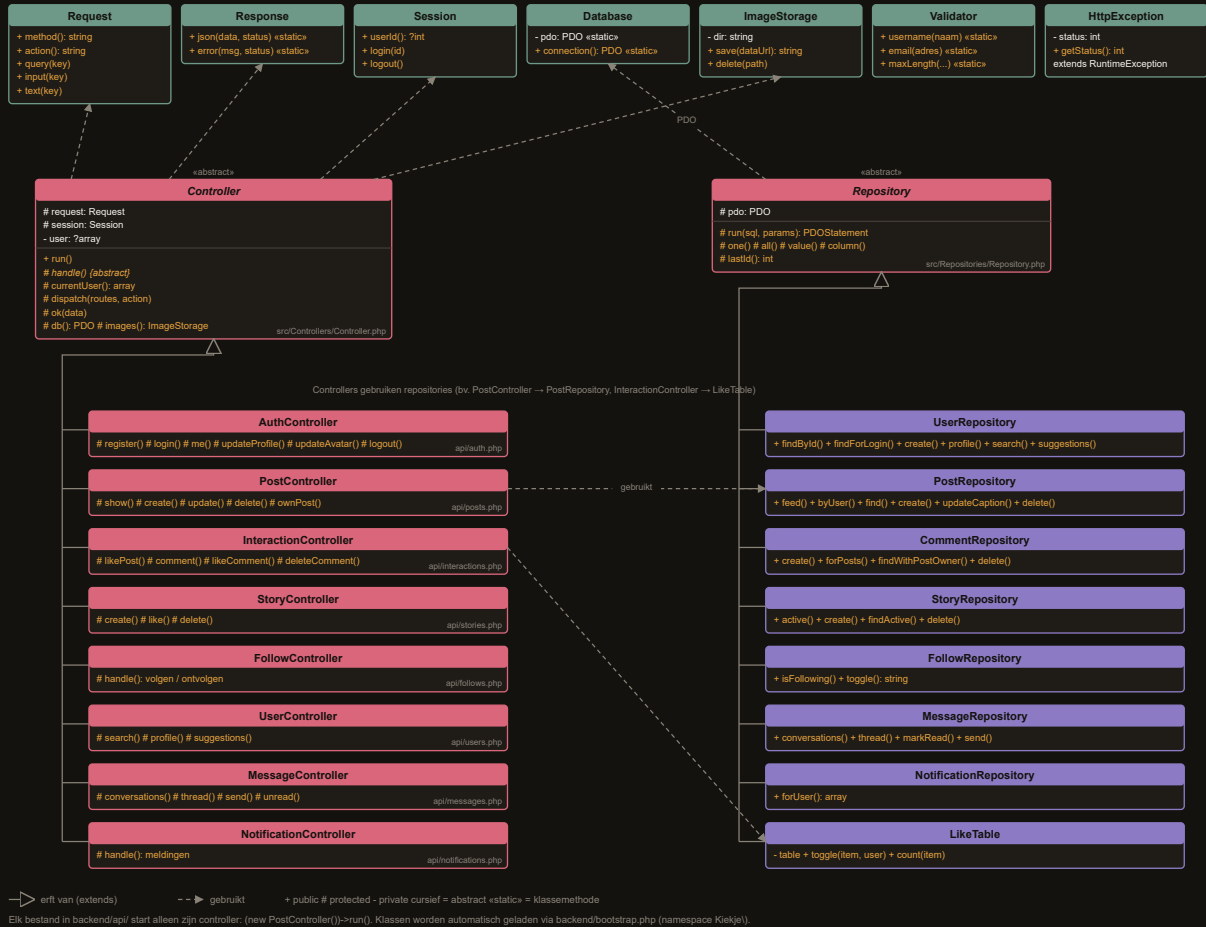
require __DIR__ . '/../bootstrap.php';                    // zet de keuken aan (laadt alle
klassen)
(new Kiekje\Controllers\PostController())->run();         // "chef-kok van de foto's, aan het
werk!"

```

## Klassendiagram — Kiekje backend

De echte PHP-klassen in backend/src/: overerving (Controller, Repository), encapsulatie en hergebruik (LikeTable)

Core (backend/src/Core/)



Het klassendiagram: bovenaan het gereedschap (Core), links de chef-koks (Controllers), rechts de voorraadbeheerders (Repositories). Het witte driehoekje betekent "erft van".

### Zeg dit

"De backend is objectgeoriënteerd. Er zijn twee **abstracte basisklassen**: **Controller** en **Repository**. Acht controllers erven van **Controller** en regelen elk één onderdeel. De repositories doen alle SQL. Zo staat elke taak op één plek, en hergebruik ik code, bijvoorbeeld met **LikeTable**."

## 9. De voorraadkast: de database

De database heet **kiekje** en heeft **9 tabellen**. Een tabel is een **lijst**, zoals een schrift met rijen.

| Tabel         | Wat staat erin?                                                           |
|---------------|---------------------------------------------------------------------------|
| users         | Alle mensen: naam, e-mail, wachtwoord (geheim gemaakt!), bio, profielfoto |
| posts         | Alle foto's: wie, welke foto, tekst, plaats                               |
| likes         | Wie welke foto leuk vindt                                                 |
| comments      | Alle reacties                                                             |
| comment_likes | Wie welke reactie leuk vindt                                              |
| follows       | Wie wie volgt                                                             |
| messages      | Alle berichtjes (en of ze gelezen zijn)                                   |
| stories       | Alle stories (24 uur zichtbaar)                                           |
| story_likes   | Wie welke story leuk vindt                                                |

### Heel simpel

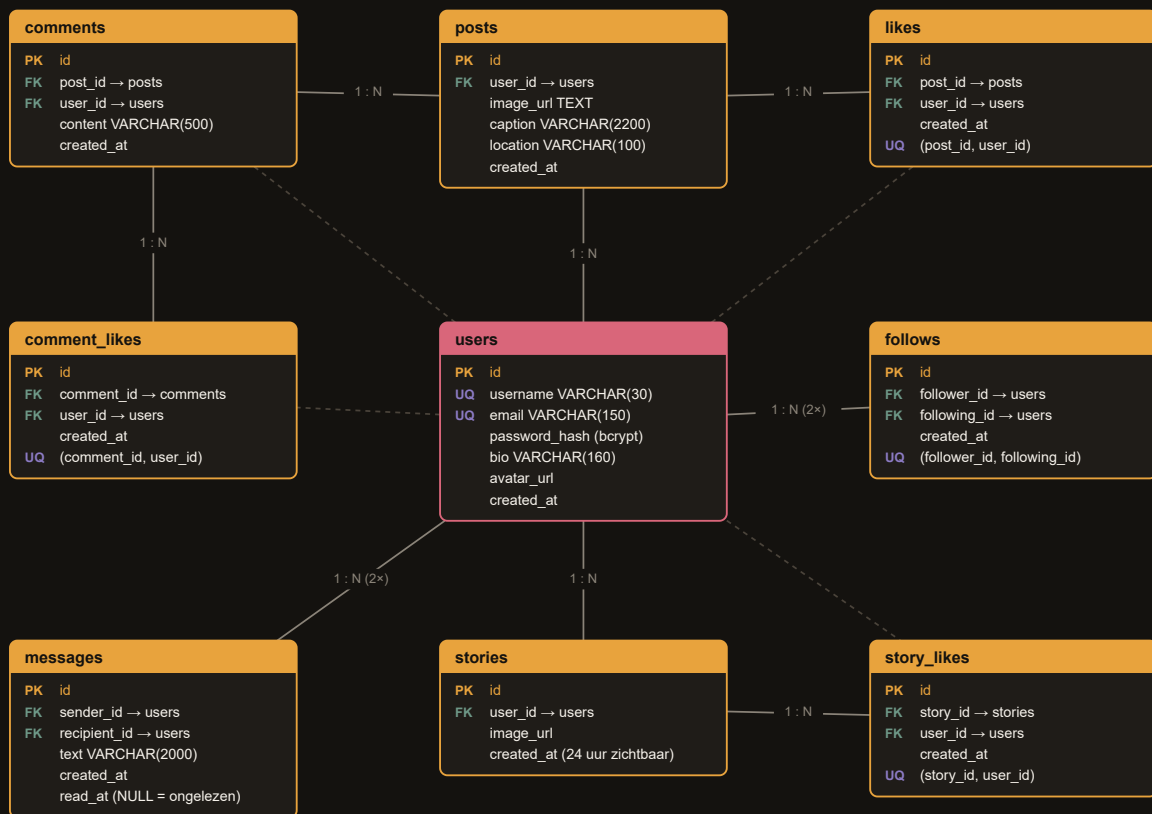
Elk ding heeft een **nummer (id)**, zoals een rugnummer bij voetbal. Een like zegt dan: "Speler 3 vindt foto 12 leuk." Zo'n nummer dat naar iets anders wijst heet een **vreemde sleutel** (foreign key).

Wordt een account verwijderd? Dan verdwijnen al zijn foto's, likes en berichtjes vanzelf mee ( `ON DELETE CASCADE` ).

**Meldingen** hebben geen eigen tabel. De app kijkt gewoon in de likes, reacties en volgers en maakt daar de meldingen van. Zo kloppen ze altijd.

## ERD — Kiekje database

Database kiekje, 9 tabellen (zie database/schema.sql)



PK = primaire sleutel FK = verwijzing (ON DELETE CASCADE) UQ = uniek (bv. één like per gebruiker per post)

Stippellijnen: user\_id-verwijzingen naar users. Meldingen hebben geen eigen tabel; ze worden afgeleid uit likes, comments, follows, story\_likes en comment\_likes.

Het ERD: alle 9 tabellen en hoe ze aan elkaar vastzitten. "1 : N" betekent: één gebruiker kan veel foto's hebben.

## 10. Zo gaat het stap voor stap

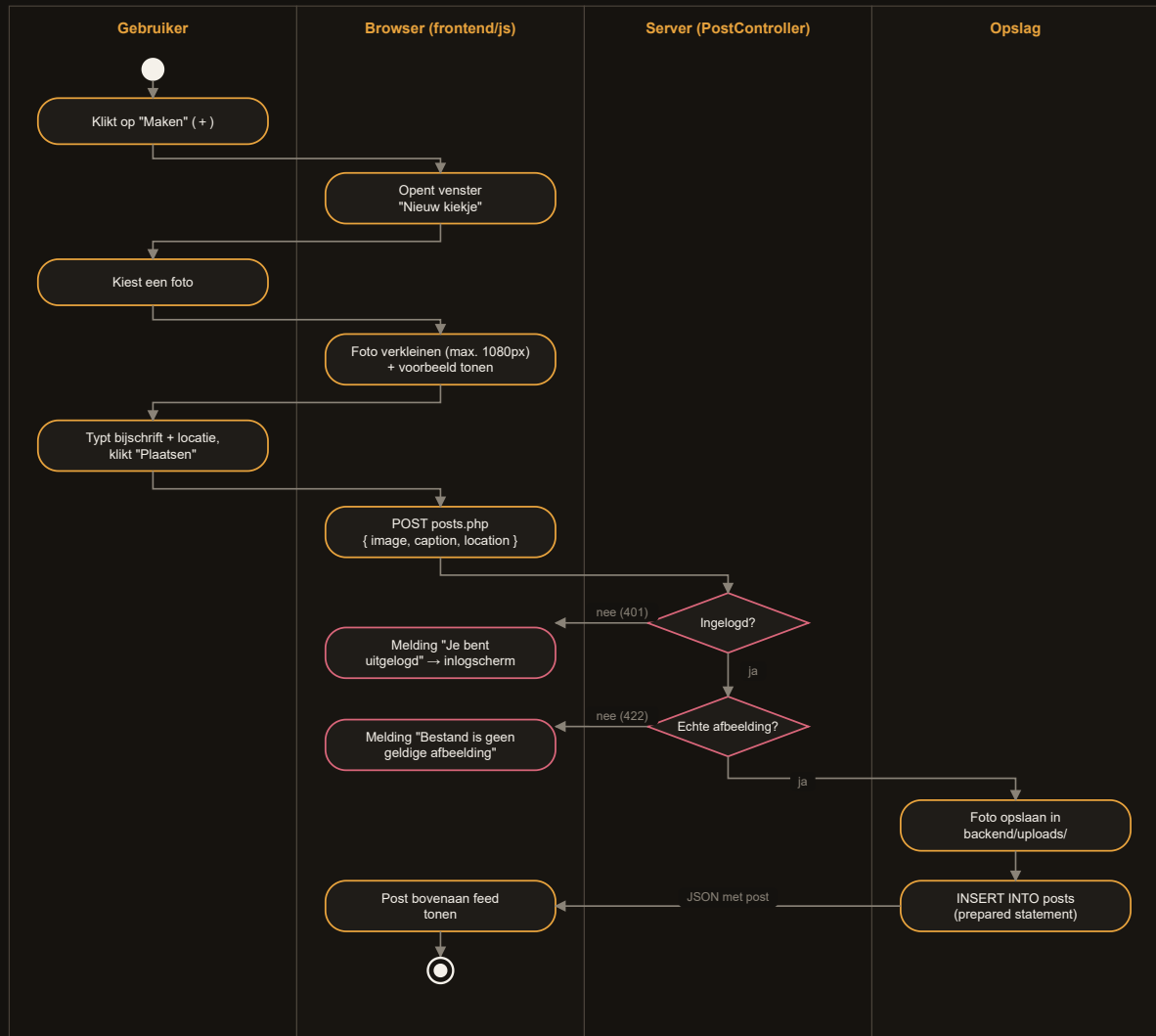
---

### Een foto plaatsen

1. Je drukt op **Maken ( + )** en kiest een foto.
2. De browser maakt de foto **kleiner** (max. 1080 pixels), zodat hij snel verstuurd wordt.
3. Je typt een tekstje en drukt op **Plaatsen**.
4. De ober brengt de foto naar `posts.php`, en de `PostController` gaat aan het werk.
5. De chef kijkt: **ben je ingelogd?** Zo nee: fout 401.
6. `ImageStorage` kijkt: **is dit echt een foto?** (en geen stout programmaatje vermomd als foto). Zo nee: fout 422.
7. De foto gaat in het fotoalbum ( `uploads/` ) met een **willekeurige naam**.
8. `PostRepository` schrijft in de database: "nieuwe foto van jou".
9. De ober komt terug en je foto staat **bovenaan de tijdlijn**. 🎉

## Activity Diagram — Kiekje plaatsen

Van klik tot post in de feed, per onderdeel (swimlanes)



Het activity diagram van precies dit verhaal, in vier banen: jij, de browser, de server en de opslag.

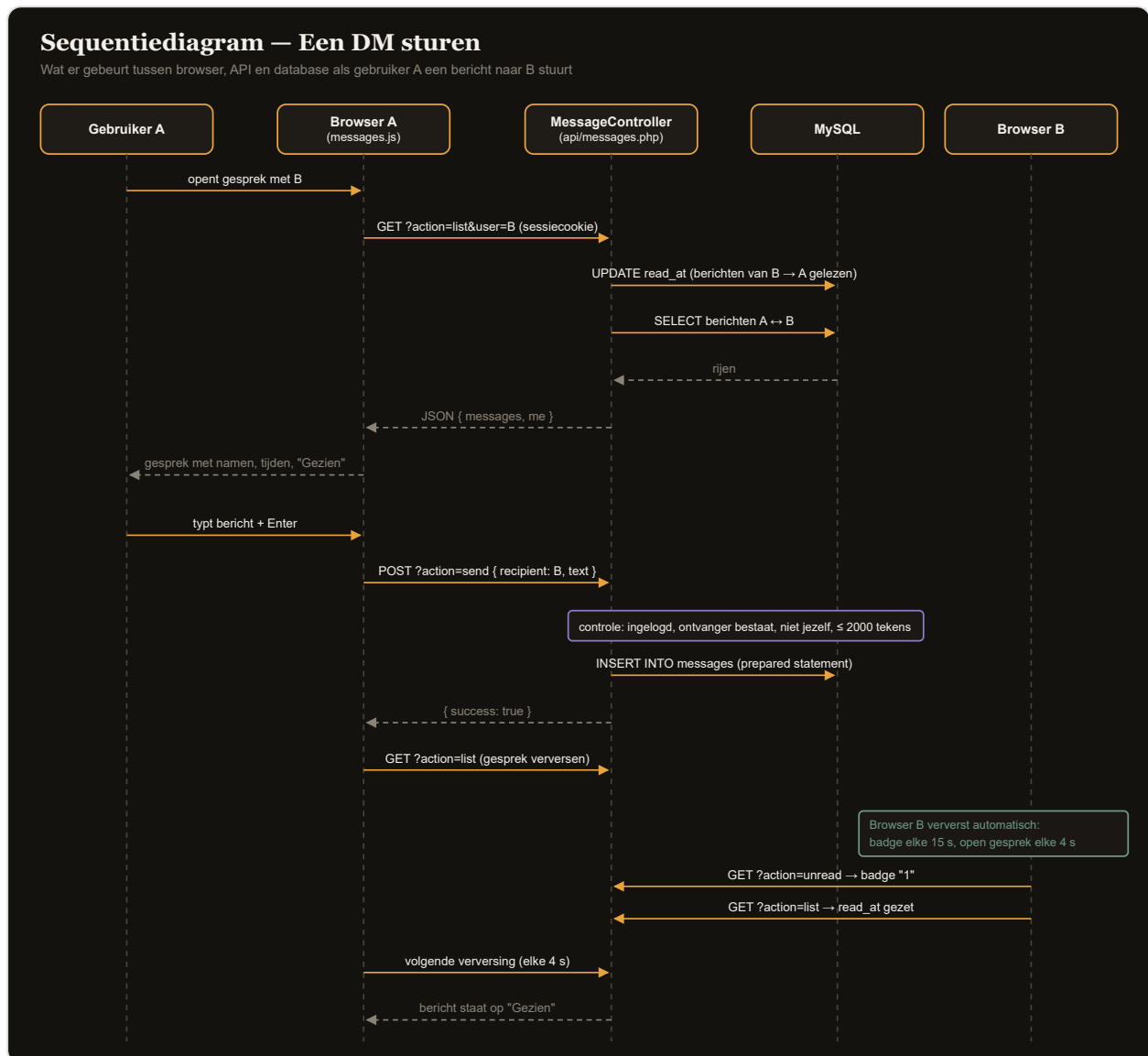
## ♥ Een foto liken

1. Je tikt op het hartje (of 2× op de foto).
2. De ober vraagt: "zet een hartje bij foto 12" ( `interactions.php?action=like` ).
3. `LikeTable` kijkt: stond er al een hartje? Dan gaat het **weg**. Zo niet, dan komt het **erbij**.
4. Het nieuwe aantal hartjes gaat terug en het scherm wordt opnieuw getekend.
5. Slim: de app stuurt nooit twee hartjes-verzoeken tegelijk. Anders zou de tweede het hartje weer weghalen.

## ■ Een berichtje sturen

1. Je opent een gesprek. De server markeert de berichten van de ander als **gelezen**.

2. Je typt "hoi" en drukt op Enter. De server controleert: ingelogd? bestaat de ontvanger? niet naar jezelf? niet te lang?
3. Het bericht gaat de database in.
4. De app van de ander kijkt **elke 4 seconden** of er iets nieuws is (bij een open gesprek) en elke 15 seconden voor het rode rondje.
5. Heeft de ander het gelezen? Dan zie jij **"Gezien"**.



Het sequentiediagram: van boven naar beneden zie je wie wanneer wat tegen wie zegt.

## 🔑 Inloggen

1. Je typt je naam (of e-mail) en wachtwoord.
2. De server zoekt jou op en vergelijkt het wachtwoord met de **geheime versie** (hash) in de database.
3. Klopt het? Dan krijg je een **armbandje** (een **sessie-cookie**), net als in een pretpark. Bij elke vraag laat de browser dat armbandje zien.

## 11. Veiligheid: de sloten op de deur

| Gevaar                                                              | Hoe beschermen we?                                                                            | Simpel                                                                                                                                                          |
|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Iemand steelt de database met wachtwoorden                          | <b>Hash &amp; salt</b> met <code>password_hash()</code> (bcrypt)                              | We bewaren het wachtwoord niet, maar een <b>soep</b> ervan. Van soep kun je de groenten niet meer terugmaken. En elke soep krijgt een eigen snufje zout (salt). |
| <b>SQL-injectie</b> : stoute tekst die de database commando's geeft | <b>Prepared statements</b> (PDO)                                                              | We geven de database een <b>invulformulier</b> . Wat jij typt komt altijd in een vakje, en wordt nooit een commando.                                            |
| <b>XSS</b> : stoute code in een reactie                             | <code>esc()</code> in de frontend, React doet het zelf                                        | Van <code>&lt;script&gt;</code> maken we gewone letters.                                                                                                        |
| Iemand past jouw foto aan                                           | Server controleert <b>eigenaarschap</b> (fout 403)                                            | Alleen jij mag aan jouw tekening kleuren.                                                                                                                       |
| Een vermomd programma als "foto"                                    | <code>ImageStorage</code> kijkt in het bestand zelf, <code>.htaccess</code> blokkeert scripts | We kijken in het cadeautje, niet alleen naar het papiertje.                                                                                                     |
| Iemand steelt je armbandje (sessie)                                 | <code>httponly</code> -cookie, nieuw sessie-id bij inloggen                                   | Je armbandje kan niet door stoute code worden gepakt, en je krijgt elke keer een nieuwe.                                                                        |
| Een andere site doet iets namens jou ( <b>CSRF</b> )                | <code>SameSite=Lax</code> + account-acties alleen via POST                                    | Je armbandje werkt alleen in ons eigen pretpark.                                                                                                                |
| Foutmeldingen verklappen geheimen                                   | Browser krijgt alleen "Serverfout."                                                           | We zeggen niet wat er in de keuken misging.                                                                                                                     |

### Zeg dit

"Wachtwoorden worden gehasht met bcrypt, met een unieke salt per wachtwoord. Tegen SQL-injectie gebruik ik overal prepared statements. De server controleert bij elke actie of je ingelogd bent en of iets echt van jou is."

## 12. Testen: de robot die alles uitprobeert

### Heel simpel

We hebben een **robot**  gemaakt die de app voor ons uitprobeert. Hij opent de browser, maakt accounts, plaatst foto's, liket, stuurt berichtjes... en kijkt of alles goed gaat. Zo hoeven wij niet elke keer alles zelf te klikken.

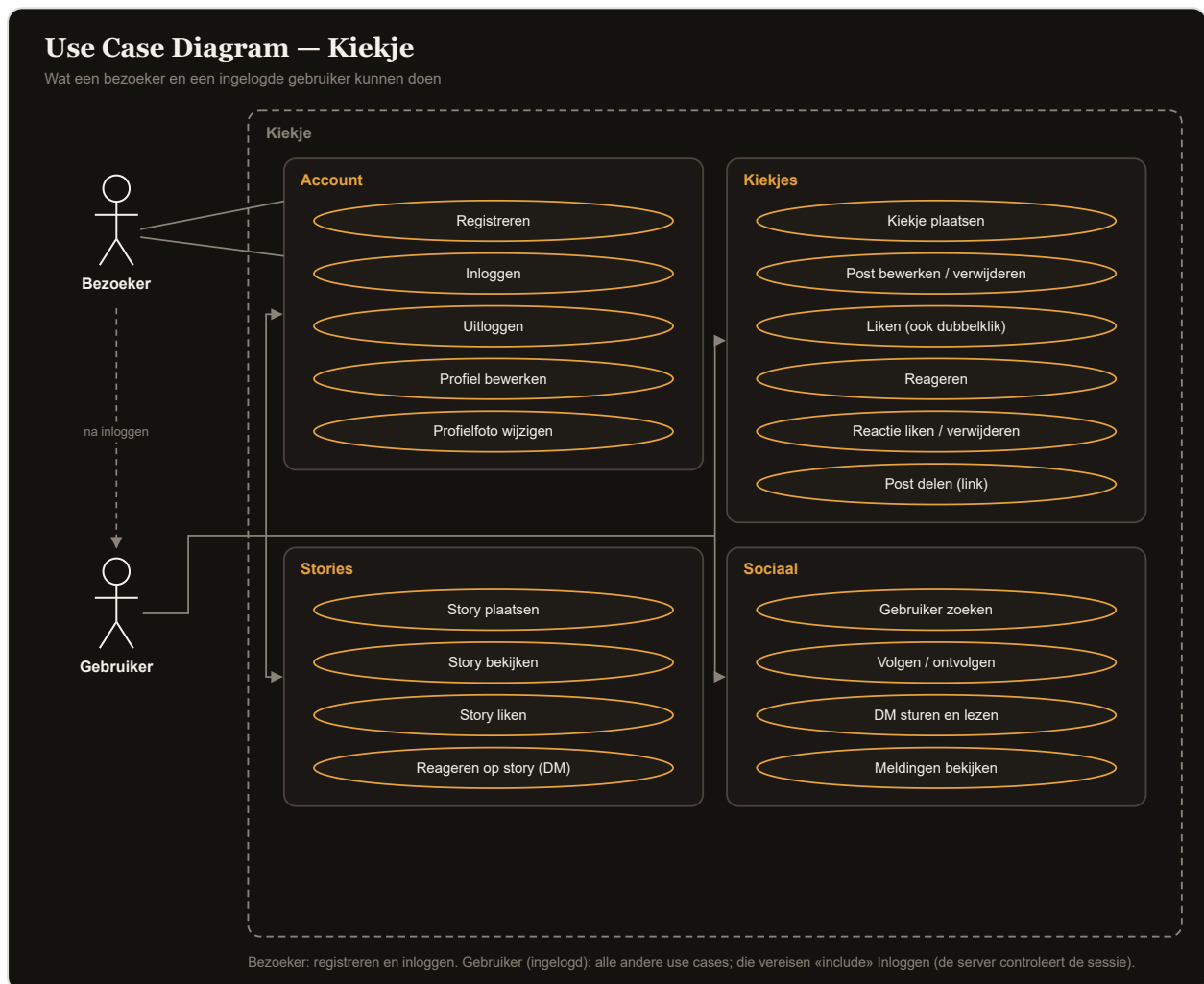
| Soort test                         | Hoeveel | Wat test het?                                                                | Starten                              |
|------------------------------------|---------|------------------------------------------------------------------------------|--------------------------------------|
| <b>Browsertests</b><br>(Puppeteer) | 9       | De hele app, zoals een echte gebruiker klikt                                 | <code>cd tests/e2e → npm test</code> |
| <b>Unit tests</b><br>(PHPUnit)     | 23      | Kleine stukjes van de keuken apart: de controleur, het fotoalbum, de oma-kok | <code>vendor/bin/phpunit</code>      |

**Belangrijk:** de robot speelt in een **aparte speeltuin**. Hij gebruikt een eigen database ( `kiekje_test` ) en een eigen kopie van de app. Zo komen er nooit test-accounts of test-foto's in de echte app. Na het testen ruimt de robot alles weer op. Dit heet **scheiding tussen test- en live-omgeving**.

## 13. De tekeningen (diagrammen)

In de map `diagrams/` staan tekeningen van de app. Ze worden gemaakt door `genereer-diagrammen.js` .

| Tekening         | Wat laat het zien? (simpel)                                     |
|------------------|-----------------------------------------------------------------|
| ERD              | De voorraadkast: alle 9 tabellen en hoe ze vastzitten           |
| Klassendiagram   | Alle koks (klassen) en wie van wie erft                         |
| Domeinmodel      | De "dingen" in de app (gebruiker, foto, reactie...) zonder code |
| Use case diagram | Wat een bezoeker en een ingelogde gebruiker kunnen doen         |
| Activity diagram | De stappen van een foto plaatsen                                |
| Sequentiediagram | Wie zegt wat tegen wie bij een berichtje                        |
| Architectuur     | Het restaurant: eetzaal, keuken, voorraadkast                   |
| Sitemap          | Alle schermen van de app                                        |
| Wireframes       | Een schets van het scherm op telefoon en computer               |



Use case diagram: wat kan iedereen doen?

## 14. Spiekbriefje voor de presentatie (± 5 minuten)

---

### 1. Wat is het? (30 sec)

"Ik heb Kiekje gemaakt: een app zoals Instagram. Je kunt foto's en stories delen, liken, reageren, mensen volgen en berichten sturen."

### 2. Laat de app zien (1,5 min)

Log in als `lisa_natuur` (wachtwoord `Kiekje2026!`). Laat zien: de feed → dubbeltik op een foto (groot hartje) → reacties openen → een story bekijken → een berichtje sturen → meldingen (belletje) → zoeken → je profiel met profielfoto bijsnijden.

### 3. Hoe zit het in elkaar? (1 min) — laat het architectuurdiagram zien

"De frontend is HTML, CSS en JavaScript, en de zoekpagina is een React-component. Die praat via een API met de backend in PHP. Alles staat in een MySQL-database met 9 tabellen — hier is het ERD."

### 4. OOP (1 min) — laat het klassendiagram zien

"De backend is objectgeoriënteerd. Een abstracte `Controller` met acht subklassen, een abstracte `Repository` met de SQL. Overerving, abstractie, encapsulatie en hergebruik met `LikeTable`."

### 5. Veiligheid en testen (1 min)

"Wachtwoorden zijn gehasht met bcrypt en een salt. Prepared statements tegen SQL-injectie. De server controleert eigenaarschap. Ik heb 23 unit tests en 9 browsertests, die in een aparte testdatabase draaien, zodat de echte data nooit geraakt wordt."

## 15. Vragen die de docent kan stellen

| Vraag                                                        | Jouw antwoord                                                                                                                                                                            |
|--------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Waarom sla je het wachtwoord niet gewoon op?                 | Als de database gestolen wordt, kunnen ze de wachtwoorden niet lezen. Ik sla alleen een <b>hash</b> op (bcrypt) met een <b>salt</b> , zodat twee dezelfde wachtwoorden toch anders zijn. |
| Wat is SQL-injectie en hoe voorkom je het?                   | Stoute tekst die de database een commando geeft. Ik gebruik <b>prepared statements</b> : de invoer gaat altijd in een apart vakje ( ? ) en wordt nooit uitgevoerd.                       |
| Wat is het verschil tussen een klasse en een object?         | Een klasse is de <b>vorm</b> (bijv. <code>PostController</code> ), een object is wat je ermee <b>maakt</b> ( <code>new PostController()</code> ).                                        |
| Waar zie je overerving?                                      | <code>class PostController extends Controller</code> . Alle 8 controllers erven <code>run()</code> en <code>currentUser()</code> van <code>Controller</code> .                           |
| Waarom is Controller abstract?                               | Je gebruikt hem nooit los; hij is een basis. Elke subklasse <b>moet</b> zelf <code>handle()</code> maken.                                                                                |
| Wat doet een Repository?                                     | Daar staat alle SQL. Zo schrijft een controller zelf geen SQL, en staat alles over de database op één plek.                                                                              |
| Waar gebruik je React?                                       | De zoekpagina: componenten <code>SearchPage</code> , <code>UserRow</code> , <code>Avatar</code> , met <code>useState</code> en <code>useEffect</code> .                                  |
| Hoe weet de server wie je bent?                              | Na inloggen krijg je een <b>sessie-cookie</b> . PHP onthoudt bij die sessie jouw gebruikers-id.                                                                                          |
| Wat gebeurt er als ik andermans foto probeer te verwijderen? | De server controleert of de foto van jou is. Zo niet: fout <b>403</b> "Dit is niet jouw post."                                                                                           |
| Hoe test je?                                                 | 9 browsertests met Puppeteer (de hele app) en 23 unit tests met PHPUnit (losse klassen). Ze draaien in een aparte testdatabase.                                                          |
| Waarom hebben meldingen geen tabel?                          | Ze worden afgeleid uit likes, reacties en volgers. Zo kunnen ze nooit verkeerd zijn: haalt iemand zijn like weg, dan verdwijnt de melding ook.                                           |
| Wat zou je nog verbeteren?                                   | Een knop om je account te verwijderen (AVG), een limiet op inlogpogingen, en locatiegegevens (EXIF) uit foto's halen op de server.                                                       |

## 16. Woordenlijst

| Woord              | Heel simpel                                               |
|--------------------|-----------------------------------------------------------|
| Frontend           | Wat je ziet (de eetzaal)                                  |
| Backend            | Wat op de server gebeurt (de keuken)                      |
| API                | De ober: de afspraken hoe browser en server praten        |
| JSON               | Een briefje met labeltjes waarin de ober praat            |
| Database           | De voorraadkast waar alles bewaard wordt                  |
| Tabel              | Een lijst in de database, zoals een schrift met rijen     |
| id / sleutel       | Het rugnummer van een ding                                |
| Klasse             | De koekjesvorm                                            |
| Object             | Het koekje                                                |
| Overerving         | Een kind erft kunstjes van zijn ouder                     |
| Abstract           | Een plan dat alleen de kinderen echt gebruiken            |
| Encapsulatie       | Een doos met een slot                                     |
| Component (React)  | Een LEGO-blokje                                           |
| State (useState)   | Het geheugentje van een blokje                            |
| render()           | De tekenaar die het scherm maakt                          |
| Sessie / cookie    | Het armbandje in het pretpark                             |
| Hash               | Soep van je wachtwoord: niet meer terug te maken          |
| Salt               | Een snufje zout, anders bij elk wachtwoord                |
| Prepared statement | Een invulformulier voor de database                       |
| Unit test          | Eén klein stukje apart testen                             |
| End-to-end test    | De robot die de hele app uitprobeert                      |
| Single-page app    | Eén pagina die van inhoud wisselt zonder opnieuw te laden |

Meer details: [README.md](#), [docs/architectuur.md](#), [docs/api.md](#), [backend/README.md](#) en [docs/documentatie.md](#). Succes met je presentatie! 🍀